

DDoS Attack Detection Using Deep Neural Networks

This report presents a deep learning approach to binary classification of network flow data, identifying DDoS attacks with high precision while maintaining a low false positive rate to avoid blocking legitimate users.

Raid Kellil

April 2026

DDoS Attack Detection Using Deep Neural Networks

University Project Report

Raid Kellil

1. Introduction

A Distributed Denial of Service (DDoS) attack aims to disrupt the normal traffic of a targeted server, service, or network by overwhelming the target or its surrounding infrastructure with a flood of Internet traffic. As attack methods grow in sophistication, traditional rule-based detection systems struggle to keep pace. Machine learning, particularly deep learning, offers a robust alternative by learning complex, non-linear patterns from network traffic data to distinguish between malicious DDoS flows and benign ones.

This project implements a Deep Neural Network (DNN) to perform binary classification on network flow data, identifying DDoS attacks with high precision while maintaining a low false positive rate to avoid blocking legitimate users.

2. Dataset and Feature Engineering

The dataset used in this project is sourced from a publicly available network traffic dataset containing realistic DDoS and normal traffic patterns. The dataset contains 22 numerical features derived from network flow statistics (e.g., packet rates, TCP flags, inter-arrival times). Crucially, the dataset includes a "stealthy" subset (25% of DDoS traffic) where attack parameters closely resemble normal traffic, adding a layer of difficulty to the classification task.

Partition	Total Samples	Normal Traffic	DDoS Traffic	DDoS Ratio
Train	20,000	16,999	3,001	15.0%
Validation	3,000	2,550	450	15.0%
Test	2,000	1,701	299	15.0%

Table 1: Dataset partition summary

Preprocessing: Neural networks are highly sensitive to the scale of input features. A StandardScaler is fitted on the training data to achieve zero mean and unit variance, and subsequently applied to the validation and test sets to prevent data leakage. Given the class imbalance (roughly 1:5.6 DDoS to Normal ratio), class weights are computed and passed to the model during training to penalize misclassifications of the minority DDoS class more heavily.

3. Methodology

A feedforward Deep Neural Network (DNN) is constructed using the Keras Sequential API. The architecture uses Dropout layers to prevent overfitting. The final Sigmoid layer outputs a

probability score between 0 and 1, representing the likelihood of a DDoS attack.

Layer Type	Units	Activation	Dropout
Input	22	-	-
Dense	64	ReLU	30%
Dense	32	ReLU	20%
Dense	16	ReLU	-
Dense (Output)	1	Sigmoid	-

Table 2: DNN architecture summary

Training Strategy:

- Loss Function: Binary Cross-Entropy
- Optimizer: Adam
- Early Stopping: Training is monitored on the validation loss with a patience of 5 epochs. The best weights are restored, preventing overfitting. Training concluded at 26 epochs.

Threshold Tuning: By default, a probability threshold of 0.5 is used for binary classification. However, in network security, the cost of a false positive (blocking legitimate traffic) is often weighed differently than a false negative (letting an attack through). We tuned the decision threshold on the validation set's ROC curve to target a False Positive Rate (FPR) of approximately 2%. This resulted in an optimal threshold of 0.7701.

4. Results and Evaluation

Evaluated on the unseen test set (2,000 samples) using the tuned threshold of 0.7701, the model achieved the following results:

Metric	Score
Accuracy	96.65%
Precision	90.00%
Recall (Detection Rate)	87.29%
F1-Score	88.62%

Specificity	98.30%
Test FPR	1.70%

Table 3: Model performance metrics on the test set

Confusion Matrix:

	Predicted Normal	Predicted DDoS
Actual Normal	1,672 (TN)	29 (FP)
Actual DDoS	38 (FN)	261 (TP)

Table 4: Confusion matrix on the test set

The model exhibits exceptional discriminative ability, achieving an Area Under the ROC Curve (AUC) of 0.9887 and an Average Precision (AP) of 0.9580.

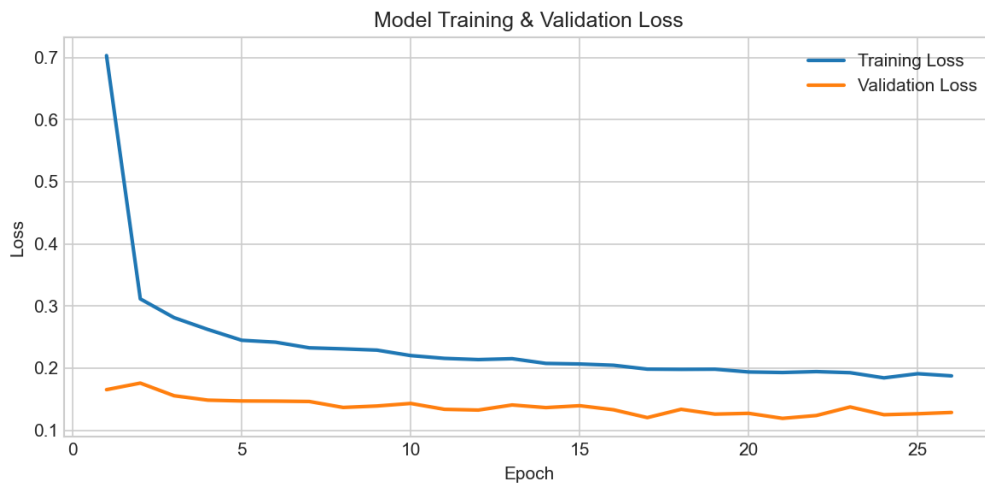


Figure 1: ROC Curve - AUC = 0.9887

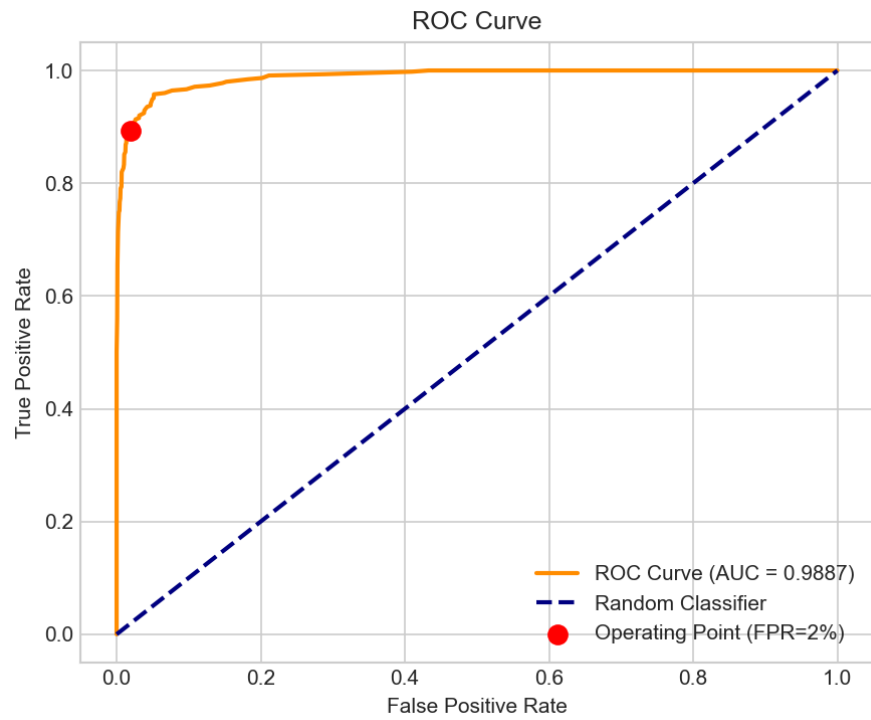


Figure 2: Precision-Recall Curve - AP = 0.9580

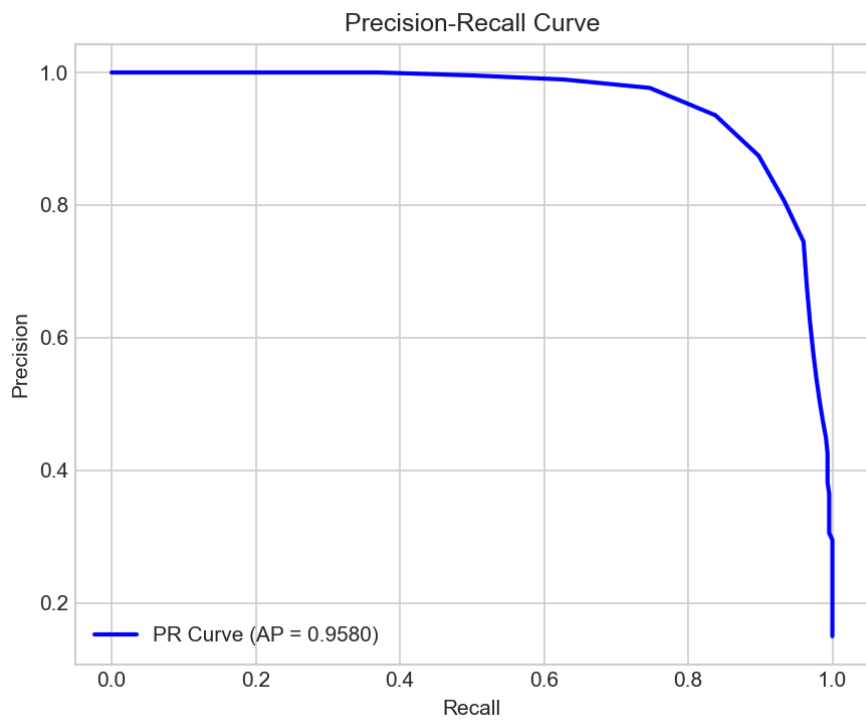


Figure 3: Training History - Loss and Accuracy

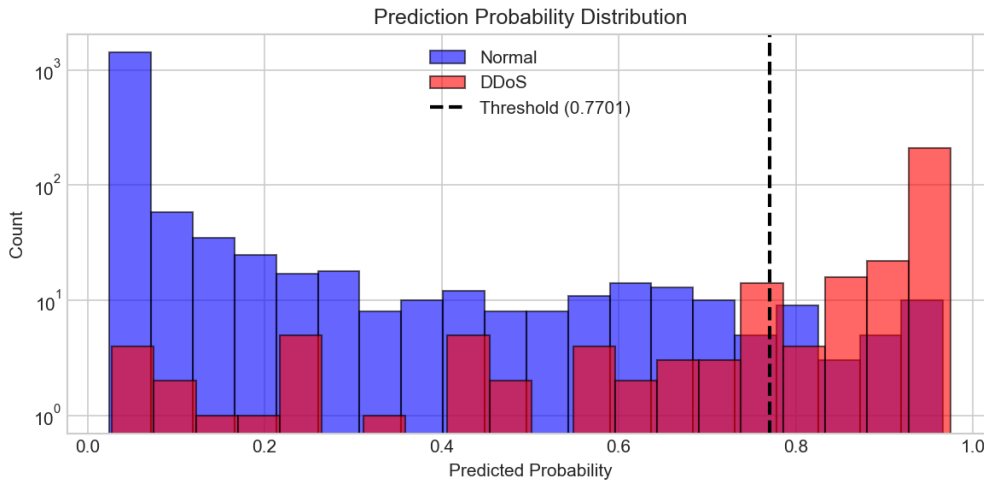


Figure 4: Histogram of Predicted Probabilities

The histogram of predicted probabilities shows a clear bimodal distribution. Normal traffic is heavily clustered near probability 0.0, while DDoS traffic is heavily clustered near 1.0. The overlap in the middle (around the 0.77 threshold) represents the "stealthy" DDoS flows that mimic normal behavior.

5. Feature Importance Analysis

To understand what drives the model's decisions, Permutation Feature Importance was calculated on the test set. The top features reveal how the model distinguishes attacks:

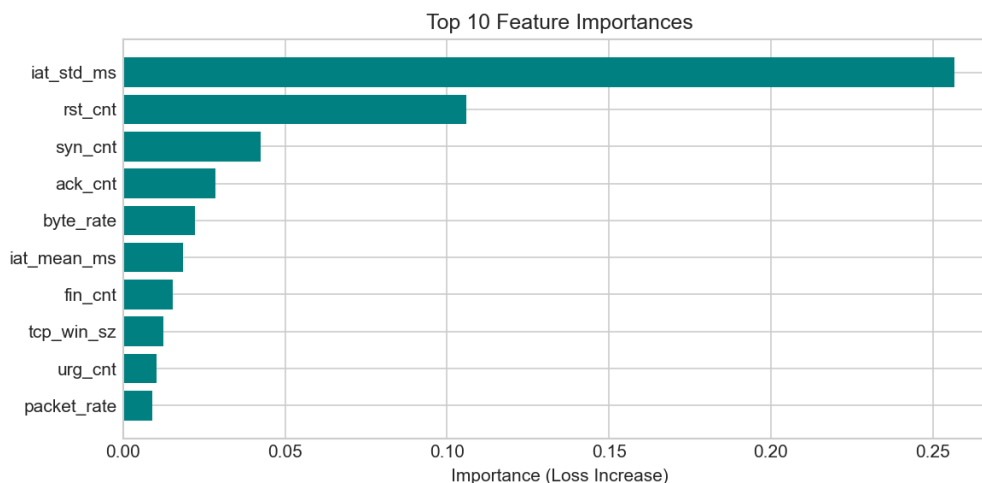


Figure 5: Permutation Feature Importance (Top 10)

1. iat_std_ms (Importance: 0.2566): The standard deviation of inter-arrival times is by far the most critical feature. DDoS attacks, particularly volumetric ones generated by scripts/botnets, send

packets at highly regular intervals (low `iat_std_ms`), whereas human-generated normal traffic is bursty and irregular.

2. `rst_cnt` (Importance: 0.1059): High reset counts are characteristic of failed or forcibly closed connections, common in DDoS floods.

3. `syn_cnt` (Importance: 0.0424): Reflects SYN flood behaviors, a classic DDoS vector.

6. Conclusion and Future Work

This project successfully demonstrates the efficacy of a Deep Neural Network in detecting DDoS attacks. By strategically tuning the classification threshold to prioritize a low False Positive Rate (1.7%), the system achieves a robust 87.3% recall while maintaining 90% precision. The feature importance analysis further aligns with domain knowledge, confirming that the model relies on meaningful network patterns such as inter-arrival time variance rather than spurious correlations.

Future Work: To improve the 38 missed DDoS instances (which largely belong to the "stealthy" class), future iterations could explore:

1. Temporal models like LSTMs or GRUs that capture time-series dependencies in packet flows.
2. Ensemble methods combining unsupervised anomaly detection with supervised classification.